

The Windows Event Logs & Forensics Playbook

A complete, detailed guide to Windows event logging, artifacts, and host-based investigation for security analysts

From Event ID 4624 to the Amcache hive, this playbook builds the artifact fluency and investigative workflow that turns a compromised Windows host into a readable timeline instead of a mystery.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 2, 2026

Preface

Windows hosts generate an enormous amount of forensic evidence by default — the challenge was never a shortage of data, it's knowing which of the thousands of daily events actually matter, and where the rest of the evidence lives outside the Event Log entirely. A SOC analyst who can recognize Event ID 4625 but doesn't know what Prefetch, Amcache, or Shimcache are is only seeing half the picture during an investigation.

This playbook covers Windows host-based investigation from both angles: the Event Log (architecture, the security events that actually matter, and PowerShell/Sysmon logging), and the broader set of forensic artifacts Windows leaves behind regardless of logging configuration — registry hives, execution artifacts, and persistence locations. The goal throughout is building a timeline: what happened, in what order, and what it means.

What this book covers

- Windows Event Log architecture: channels, providers, and .evtx files
- The core security Event IDs worth knowing by number
- Logon and authentication events, including logon types
- Process execution logging and Sysmon
- Object access, file, and registry auditing
- PowerShell logging: script block, module, and transcription logs
- Forensic artifacts beyond the Event Log: Prefetch, Amcache, Shimcache, registry hives, Jump Lists
- Building an investigation timeline from multiple evidence sources
- Common persistence mechanisms and how to detect them
- Log collection and centralization for SOC visibility
- Practical investigation recipes for common scenarios

Swastik Sagar

Contents

Preface	i
1 Introduction to Windows Forensics	1
1.1 Two kinds of evidence	1
1.2 Locard's exchange principle, applied to Windows	1
2 Windows Event Log Architecture	2
2.1 Channels	2
2.2 The .evtx file format	2
2.3 Audit policy	2
2.4 Querying events	3
3 Core Security Event IDs	4
3.1 Why memorize event IDs at all	4
4 Logon and Authentication Events	5
4.1 Logon types	5
4.2 Reading a 4625 for brute-force detection	5
4.3 Kerberos events	5
5 Process Execution and Sysmon	7
5.1 Event 4688: process creation	7
5.2 Why Sysmon anyway	7
5.3 Reading a process creation chain	7
6 Object Access, File, and Registry Events	9
6.1 File and object auditing	9
6.2 Registry auditing	9
7 PowerShell Logging	10
7.1 Why PowerShell needs its own chapter	10
7.2 The three logging types	10
7.3 Suspicious PowerShell indicators	10

8	Windows Forensic Artifacts	11
8.1	Execution artifacts	11
8.2	File and folder access artifacts	11
8.3	Key registry hive locations	12
9	Building an Investigation Timeline	13
9.1	Why timelines, not isolated events	13
9.2	Time zone and clock skew	13
9.3	Tools for timeline building	13
10	Persistence Mechanisms and Detection	14
10.1	Common persistence categories	14
10.2	Detecting WMI persistence	14
11	Log Collection and Centralization	15
11.1	Why local logs aren't enough	15
11.2	Common collection approaches	15
12	Practical Investigation Recipes	16
12.1	Suspected brute force against a specific account	16
12.2	Finding all processes spawned by an Office application	16
12.3	Checking Prefetch for evidence of execution	16
12.4	Pulling every 4688 command line containing a keyword	16
12.5	Quick triage checklist for a suspected compromised host	16

Introduction to Windows Forensics

1.1 Two kinds of evidence

Windows investigation draws on two overlapping but distinct evidence sources, and confusing them is a common early mistake.

- **Event Logs** — a structured, purpose-built logging system; what gets recorded depends entirely on audit policy configuration
- **Forensic artifacts** — side effects of normal OS operation (registry entries, cached files, execution records) that exist regardless of logging configuration, and are far harder for an attacker to fully suppress

Why both matter

A well-configured SIEM pipeline covers Event Logs well, but a host that was compromised before logging was configured correctly (or where logs were cleared) still has artifacts sitting on disk. Knowing both is what lets an investigation continue when the "obvious" evidence source is gone.

1.2 Locard's exchange principle, applied to Windows

Every process execution, file access, and network connection leaves some trace on a Windows system — the practical skill is knowing where each type of trace lives and how long it persists before being overwritten or rotated out.

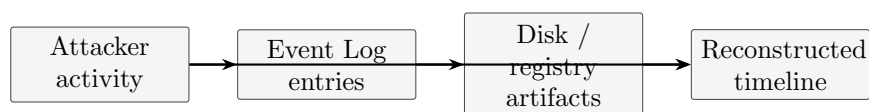


Figure 1.1. *Every action produces evidence through two parallel channels – logs and artifacts – both feeding the same reconstructed timeline.*

Windows Event Log Architecture

2.1 Channels

Windows organizes events into named channels. The three always present on every system are the classic ones, alongside many application- and service-specific channels added by installed software.

Channel	Contents
Security	Authentication, authorization, object access – the core forensic channel, requires audit policy to populate fully
System	OS and driver-level events: service start/stop, boot events
Application	Events logged by installed applications
Microsoft-Windows-Sysmon/Operational	Sysmon's detailed process, network, and file events, if installed
Microsoft-Windows-PowerShell/Operational	PowerShell script block and module logging

2.2 The .evtx file format

Each channel is backed by a binary **.evtx** file on disk, normally under `C:\Windows\System32\winevt\Logs\`. These files are size-capped and roll over (oldest events are overwritten) once full, which is why timely collection into a SIEM matters — local retention alone is often only days on a busy host.

Common local log locations

```
C:\Windows\System32\winevt\Logs\Security.evtx C:\Windows\System32\winevt\Logs\System.evtx C:\Windows\System32\winevt\Logs\Microsoft-Windows-Sysmon%40operational.evtx
```

2.3 Audit policy

Security channel content depends entirely on **audit policy** — without the right audit subcategories enabled, many of the events in this book simply never get generated, regardless of how carefully you search.

Viewing current audit policy

```
auditpol /get /category:*
```

The default-logging gap

Out-of-the-box Windows audit policy is deliberately conservative to limit log volume. Process creation with command-line logging, in particular, requires explicitly enabling both an audit subcategory and a Group Policy setting – one of the most common gaps found during a post-incident review.

2.4 Querying events

PowerShell: basic Get-WinEvent usage

```
Get-WinEvent -LogName Security -MaxEvents 50 Get-WinEvent -FilterHashtable  
@LogName='Security'; Id=4625 -MaxEvents 20 Get-WinEvent -Path C:\evidence\  
Security.evtx
```

wevtutil, the command-line log utility

```
wevtutil qe Security /c:20 /rd:true /f:text wevtutil epl Security C:\export\  
Security.evtx
```

Core Security Event IDs

3.1 Why memorize event IDs at all

Recognizing an ID on sight during triage is far faster than looking it up mid-investigation. These are the ones worth knowing cold.

ID	Meaning
4624	Successful logon
4625	Failed logon
4634 / 4647	Logoff (system-initiated / user-initiated)
4648	Logon using explicit credentials (runas, lateral movement indicator)
4672	Special privileges assigned to new logon (admin-equivalent logon)
4688	New process created
4697	Service installed
4698	Scheduled task created
4720	User account created
4728 / 4732	Member added to a security-enabled global / local group
4738	User account changed
5140	Network share accessed
1102	Audit log cleared (Security channel)
7045	New service installed (System channel)

The single most important event on this list

Event 1102 — audit log cleared — is close to a direct admission that someone is trying to cover their tracks. Any well-tuned SIEM should treat this as a high-priority alert regardless of who triggered it.

Logon and Authentication Events

4.1 Logon types

Event 4624/4625's **Logon Type** field is one of the highest-value fields in the entire Security channel — it tells you *how* the authentication happened, which shapes the entire interpretation of the event.

Type	Meaning
2	Interactive – physical console logon
3	Network – e.g. accessing a share, common and usually benign at volume
4	Batch – scheduled task execution
5	Service – a service starting under an account
7	Unlock – workstation unlocked
8	NetworkCleartext – credentials sent in cleartext over the network
9	NewCredentials – e.g. <code>runas /netonly</code>
10	RemoteInteractive – RDP logon
11	CachedInteractive – logon using cached domain credentials, no DC contact

Type 10 at odd hours

A Type 10 (RDP) logon from an account that never normally uses RDP, or at an hour outside that user's pattern, is one of the highest-value simple detections in the entire Security channel – cheap to build, high signal.

4.2 Reading a 4625 for brute-force detection

- **Failure Reason / Sub Status** – distinguishes bad password from account locked, disabled, or expired
- **Source Network Address** – where the attempt came from
- **Account Name** – which account was targeted
- High volume of 4625 across many different Account Names from one source is credential spraying; high volume against one account is brute force

4.3 Kerberos events

ID	Meaning
4768	Kerberos TGT (Ticket Granting Ticket) requested
4769	Kerberos service ticket requested
4771	Kerberos pre-authentication failed

Kerberoasting signature

A burst of 4769 events requesting RC4 (not AES) service tickets for multiple different service accounts from one source, especially outside normal hours, is the classic Kerberoasting pattern – an attacker requesting ticket hashes to crack offline.

Process Execution and Sysmon

5.1 Event 4688: process creation

With the right audit policy and Group Policy setting enabled, 4688 logs every new process, including its command line — one of the single most valuable event types for any investigation, since it directly shows what was actually executed.

Enabling command-line logging in 4688

```
-- Group Policy:  -- Computer Configuration > Administrative Templates > System >
-- Audit Process Creation > "Include command line in process creation events"
```

5.2 Why Sysmon anyway

Sysmon (System Monitor, a free Microsoft Sysinternals tool) captures far more detail than native 4688 out of the box: full process trees, network connections tied to their originating process, file creation hashes, and more, all in one dedicated channel.

Event ID	Captures
1	Process creation, including hash and full command line
3	Network connection, tied to the originating process
5	Process terminated
7	Image/DLL loaded
8	CreateRemoteThread – classic process injection indicator
10	ProcessAccess – one process opening a handle to another
11	File created
12/13/14	Registry key/value created, modified, or renamed
22	DNS query

Sysmon config matters as much as Sysmon itself

Sysmon logs nothing meaningful without a well-tuned configuration file defining which events and fields to capture – SwiftOnSecurity's public Sysmon config is the most widely used community starting point.

5.3 Reading a process creation chain

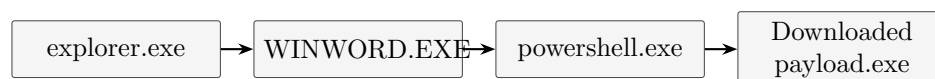


Figure 5.1. *A classic malicious document chain: a Word macro spawning PowerShell, which spawns a downloaded payload – each link visible via parent/child process fields.*

Suspicious parent/child pairs

Office applications (WINWORD.EXE, EXCEL.EXE) spawning `cmd.exe`, `powershell.exe`, or `wscript.exe` is one of the highest-value detection patterns available – legitimate documents essentially never do this.

Object Access, File, and Registry Events

6.1 File and object auditing

ID	Meaning
4656	Handle to an object requested
4663	Attempt to access an object (the actual access, following 4656)
4660	Object deleted
4670	Permissions on an object changed

Object auditing requires both the relevant audit subcategory *and* a System Access Control List (SACL) configured on the specific file, folder, or registry key of interest – it is not enabled globally by default, for performance reasons.

6.2 Registry auditing

Sysmon registry events, matched against common persistence keys

Event ID 13: RegistryEvent (Value Set) TargetObject: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run

Registry as a first stop, not just for persistence

Beyond persistence, the registry also records installed software (via Uninstall keys), USB device history, and recently opened files – making it one of the richest single artifact sources on a Windows host.

PowerShell Logging

7.1 Why PowerShell needs its own chapter

PowerShell is both a legitimate administration tool and the single most common post-exploitation execution method on Windows – fileless attacks that never touch disk as a traditional executable routinely rely on it, making PowerShell-specific logging disproportionately valuable.

7.2 The three logging types

Type	Captures
Module logging	Records pipeline execution details for specified modules (Event ID 4103)
Script block logging	Records the full de-obfuscated content of executed script blocks (Event ID 4104) – the single most valuable of the three
Transcription	Writes a full text transcript of a PowerShell session to a file on disk

Enabling script block logging via Group Policy path

Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell > Turn on PowerShell Script Block Logging

Why script block logging specifically matters

Attackers routinely obfuscate PowerShell with Base64 encoding or string manipulation specifically to evade simple keyword searches. Script block logging captures the code *after* PowerShell's own engine de-obfuscates it internally, which defeats most casual obfuscation attempts at the logging layer.

7.3 Suspicious PowerShell indicators

- `-EncodedCommand` / `-enc` flags, indicating Base64-encoded script content
- `-WindowStyle Hidden` or `-NoProfile -NonInteractive` combined with network activity
- IEX (Invoke-Expression) combined with a download cmdlet like `Invoke-WebRequest` or `Net.WebClient`
- `-ExecutionPolicy Bypass`, overriding the configured script execution policy

Windows Forensic Artifacts

8.1 Execution artifacts

These record that a program *ran*, independent of the Event Log entirely – extremely valuable when logging was disabled or events have already rolled off.

Artifact	What it shows
Prefetch	Files under <code>C:\Windows\Prefetch</code> , one per executable, recording run count and last-run times (up to 8 timestamps on modern Windows)
Amcache	Registry hive (<code>Amcache.hve</code>) recording executed binaries with SHA-1 hashes, install paths, and first-run time
Shimcache / AppCompatCache	Records file existence and modification time for executed and non-executed binaries, evidence order matters for interpretation
BAM/DAM	Background/Desktop Activity Moderator registry keys recording last execution time per binary, per user
UserAssist	Registry keys (ROT13-encoded) tracking GUI-launched programs and run counts

Why so many overlapping artifacts

Every one of these was built for a different OS subsystem's own purpose (performance caching, application compatibility, power management), not for forensics specifically – which is exactly why they survive independently and why cross-referencing several of them strengthens a finding.

8.2 File and folder access artifacts

Artifact	What it shows
LNK files	Shortcuts auto-created for recently opened files, containing original path, timestamps, and volume serial number
Jump Lists	Per-application recent-file lists, stored under <code>AppData\Roaming\Microsoft\Windows\Recent</code>
\$MFT	The Master File Table – the NTFS metadata record of every file, including deleted-but-not-overwritten entries
\$LogFile / \$UsnJrnl	NTFS transaction and change journals, recording file system operations even after the files themselves are gone

8.3 Key registry hive locations

Hive	Disk location
SYSTEM	C:\Windows\System32\config\SYSTEM
SOFTWARE	C:\Windows\System32\config\SOFTWARE
SAM	C:\Windows\System32\config\SAM
NTUSER.DAT	C:\Users\{user}\NTUSER.DAT – per-user hive

Building an Investigation Timeline

9.1 Why timelines, not isolated events

A single suspicious event rarely tells the full story. Ordering evidence from multiple sources — Event Log, Prefetch, registry, file system metadata — into one chronological timeline is what turns scattered clues into a coherent narrative of what actually happened.

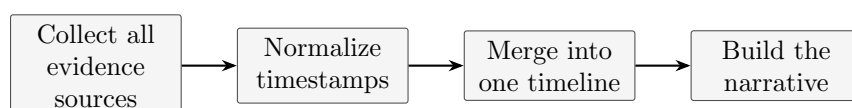


Figure 9.1. *Timeline construction is a normalization problem as much as a data-gathering one — every source’s timestamp format and time zone must agree before merging.*

9.2 Time zone and clock skew

- Event Logs store timestamps in UTC internally, but tools may display them in local time — always confirm which before comparing across sources
- Client clock drift is real; cross-reference against a trusted external time source (e.g. a proxy log or DNS server) when precision matters
- File system timestamps (MACB: Modified, Accessed, Changed, Born) can be manipulated by an attacker with sufficient privileges — treat them as corroborating evidence, not sole proof

9.3 Tools for timeline building

Tool	Role
Plaso / log2timeline	Parses many artifact types into one super-timeline
Timeline Explorer (Eric Zimmerman)	GUI viewer for large CSV timelines, with fast filtering
KAPE	Rapid, targeted artifact collection, often the first step before timeline building
Event Log Explorer / Full Event Log View	Faster, more filterable alternatives to the native Event Viewer

Persistence Mechanisms and Detection

10.1 Common persistence categories

Category	Example
Run keys	HKLM/HKCU...\CurrentVersion\Run and RunOnce
Scheduled tasks	Created via <code>schtasks</code> or the Task Scheduler, logged as Event 4698
Services	A new or modified service configured to run a malicious binary, Event 7045/4697
Startup folder	Files placed in a user's or all-users' Startup folder
WMI event subscriptions	Fileless persistence triggered by a WMI event filter, notoriously easy to overlook
DLL search order hijacking	A malicious DLL placed where a legitimate application will load it first
Browser extensions	Malicious or compromised extensions surviving reboots and even some remediation steps

Autoruns as a starting point

Sysinternals Autoruns enumerates essentially every documented persistence location in one pass, cross-referenced against a known-good hash database – almost always the fastest first step when persistence is suspected but not yet located.

10.2 Detecting WMI persistence

Enumerating WMI event subscriptions

```
Get-WmiObject -Namespace root\subscription -Class __EventFilter Get-WmiObject
-Namespace root\subscription -Class __EventConsumer Get-WmiObject -Namespace root\
subscription -Class __FilterToConsumerBinding
```

Log Collection and Centralization

11.1 Why local logs aren't enough

Local `.evtx` retention is often just days on a busy endpoint, and an attacker with sufficient privilege can clear the log outright (Event 1102) – centralizing logs off-host, ideally in near-real-time, is what actually protects the evidence.

11.2 Common collection approaches

Method	Notes
Windows Event Forwarding (WEF)	Native, agentless (uses WinRM), centralizes to a Windows Event Collector
Sysmon + a log shipper	Wazuh agent, Splunk Universal Forwarder, or similar shipping Sysmon's channel specifically
Group Policy audit configuration	Ensures consistent audit policy fleet-wide, a prerequisite for useful centralized logs

Forward before you need it

The single biggest gap in most real incidents isn't a missing detection rule – it's discovering the relevant host's logs were never centralized in the first place, and rolled off locally days before anyone looked.

Practical Investigation Recipes

12.1 Suspected brute force against a specific account

PowerShell

```
Get-WinEvent -FilterHashtable @LogName='Security'; Id=4625 | Where-Object  
$_.Properties[5].Value -eq "targetuser" | Select-Object TimeCreated,  
@n='SourceIP';e={$_.Properties[19].Value}
```

12.2 Finding all processes spawned by an Office application

Sysmon Event ID 1, filtered by parent image

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" | Where-Object $_.Id  
-eq 1 -and $_.Message -match "WINWORD.EXE"
```

12.3 Checking Prefetch for evidence of execution

Using PECmd, Eric Zimmerman's Prefetch parser

```
PECmd.exe -d C:\Windows\Prefetch --csv C:\output
```

12.4 Pulling every 4688 command line containing a keyword

PowerShell

```
Get-WinEvent -FilterHashtable @LogName='Security'; Id=4688 | Where-Object  
$_.Message -match "powershell.*-enc"
```

12.5 Quick triage checklist for a suspected compromised host

- Pull 4624/4625 for the account in question, note logon types and source IPs
- Check 4688 / Sysmon Event 1 for unusual parent/child process pairs
- Run Autoruns to check for new persistence entries
- Check Prefetch and Amcache for execution evidence covering the suspected window
- Review PowerShell script block logs (4104) for encoded or obfuscated commands
- Check for Event 1102 (log cleared) as a sign of anti-forensic activity